

Embedded IoT Systems

Homework

Question 1: ESP32 Webserver (webserver.cpp)

Part A: Short Questions

1. **WebServer server(80)** creates an HTTP server on the ESP32. Port 80 is the default port used by web browsers for HTTP communication.
2. **server.on("/", handleRoot)** links the root URL of the web server to the handleRoot function, which generates and sends the webpage to the client.
3. **server.handleClient()** is placed in the loop to continuously listen and respond to client requests. If removed, the server will not respond to any browser requests.
4. **server.send(200, "text/html", html)** sends a successful HTTP response (status 200) with HTML content to the client browser.
5. Displaying previously stored sensor values is faster and avoids frequent DHT sensor access, which can cause delays or incorrect readings if read repeatedly.

Part B: Long Question

The ESP32 first connects to a Wi-Fi network using SSID and password and receives an IP address via DHCP. A web server is initialized on port 80 and listens for HTTP requests. When a user opens the ESP32 IP address in a browser, the server calls the handleRoot function to display a dynamic webpage. A push button is used to trigger DHT sensor readings. The temperature and humidity values are displayed on an OLED and updated on the webpage. Meta refresh is used to automatically reload the webpage for real-time monitoring. Common issues include Wi-Fi connection failure, NaN sensor readings, blocking delays, and OLED display errors, which are solved by correct configuration, delays between sensor reads, and non-blocking code.

Question 2: Blynk Cloud Interfacing (blynk.cpp)

Part A: Short Questions

1. The Blynk Template ID identifies the cloud project and must match the firmware to allow proper connection.
2. Template ID links the project, while the Auth Token uniquely authenticates each device to the Blynk Cloud.
3. Using DHT22 code for a DHT11 sensor causes incorrect readings due to differences in resolution

and timing requirements.

4. Virtual Pins are software-based pins in Blynk used for cloud communication and are preferred because they are flexible and independent of physical GPIO pins.

5. BlynkTimer is used instead of delay() to prevent blocking code execution and maintain a stable cloud connection.

Part B: Long Question

A Blynk template is created with datastreams for temperature and humidity. The ESP32 firmware uses the Template ID, Template Name, and Auth Token to connect securely to the Blynk Cloud. The DHT sensor is configured correctly, and data is read at fixed intervals. Temperature and humidity values are sent to the cloud using Blynk.virtualWrite(), updating the dashboard widgets in real time. Common problems such as device offline status, incorrect values, and dashboard issues are resolved by verifying tokens, sensor type, and virtual pin configuration.

